

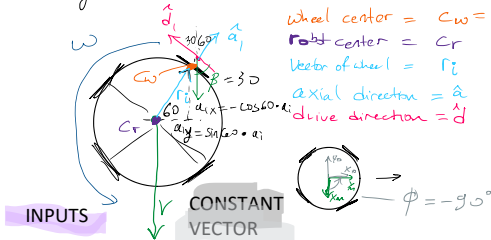
williams_model.py: calculates:

- wheel contact velocity, - d and a wheel slip velocities
- Wheel force from wheel slip velocities, friction coefficients assumed

① One omni-wheel contact

- robot velocity = V
- robot body angular velocity = ω

② Geometry



wheel center = C_w
 robot center = C_r
 vector of wheel = r_i
 axial direction = $\hat{a}$
 drive direction = $\hat{d}$

③ Velocity of wheel center

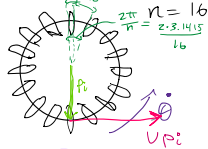
$$V_{contact} = V + \omega \times r_i$$

wheel radius vector = P_i

④ Contact point velocity

$$V_{contact} = V_{center} + V_{P_i}$$

wheel angular velocity = $\dot{\theta}$
 peripheral wheel speed = $V_{P_i} = \dot{\theta} \times P_i$



⑤ Projection of contact velocity onto axes

$$V_{slip,d} = V_{contact} \cdot \hat{d}_i$$

V_{P_i} is same axis as $\hat{d}_i$ $\therefore$ dot product with itself is 1

$$= (V + (\omega \times r_i) + (\dot{\theta} \times P_i)) \cdot \hat{d}_i$$

scalar

$$= V \cdot \hat{d}_i + (\omega \times r_i) \cdot \hat{d}_i + \dot{\theta} \cdot P_i$$

$$V_{slip,a} = V_{contact} \cdot \hat{a}_i$$

$$= (V + (\omega \times r_i) + (\dot{\theta} \times P_i)) \cdot \hat{a}_i$$

$\hat{a}$ direction

$$= V \cdot \hat{a}_i + (\omega \times r_i) \cdot \hat{a}_i + (\dot{\theta} \times P_i) \cdot \hat{a}_i$$

so the slip velocities (drive and axial directions) are calculated as follows.

$$V_{slip,d} = V \cdot \hat{d}_i + (\omega \times r_i) \cdot \hat{d}_i + \dot{\theta} \cdot P_i$$

$$V_{slip,a} = V \cdot \hat{a}_i + (\omega \times r_i) \cdot \hat{a}_i + \dot{\theta} \cdot P_i$$

$$\begin{bmatrix} i & j & k \\ 0 & 0 & \omega \\ a_{ix} & a_{iy} & 0 \end{bmatrix} = i(-a_{iy}\omega) - j(-a_{ix}\omega) = \begin{bmatrix} -\omega a_{iy} \\ \omega a_{ix} \\ 0 \end{bmatrix}$$

these give scalar numbers in the $\hat{d}$ directions.

⑤ Force generated by each wheel at point P_i

$$F_i = -\frac{m g}{n_{wheels}} \left(\mu_d(V_{slip,d}) \cdot \hat{d} + \mu_a(V_{slip,a}) \cdot \hat{a} \right) = -\frac{2.698 \text{ kg} \cdot 9.81}{4} \left(\mu_d(V_{slip,d}) \cdot \begin{bmatrix} \hat{d}_x \\ \hat{d}_y \\ \hat{d}_z \end{bmatrix} + \mu_a(V_{slip,a}) \cdot \begin{bmatrix} \hat{a}_x \\ \hat{a}_y \\ \hat{a}_z \end{bmatrix} \right)$$

4 wheels where $\mu_d(v)$, $\mu_a(v)$ are functions of the drive

and axial components of friction dependent on velocity.

* Need to experimentally determine $\mu_d(v)$, $\mu_a(v)$

Williams 2002 approximates $\mu_d(v)$ and $\mu_a(v)$ to the following:

$$\mu_d(v_{slip,d}) = \mu_{d,max} \frac{2}{\pi} \arctan(K v_{slip,d})$$

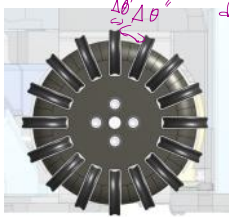
$$\mu_a(v_{slip,a}) = \mu_{a,max} \frac{2}{\pi} \arctan(K v_{slip,a})$$

where $\mu_{d,max}$, $\mu_{a,max}$, and K are constants.

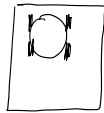
* Williams chose $K=1000$ for numerical stability in simulations

* important note. Williams provides an improved friction model, however their wheel has half the rollers as ours and the plastic material touches the floor in between.

∴ we do not need the improved friction model as our wheels do not touch the wheel material in between before the next roller!



Estimate $\mu_{d,max}$ → all wheels aligned in common direction
ie



rigid board with carpet



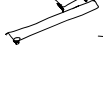
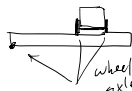
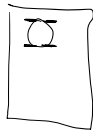
SLIP



$$\mu_{d,max} = \tan \theta_d$$

wheel axes parallel to pivoting edge

Estimate $\mu_{a,max}$ → all wheels perpendicular to surface



SLIP



$$\mu_{a,max} = \tan \theta_a$$

robot_dynamics.py:
- Wheel torques
- x,y,phi accelerations

⑥ Dynamic Equations (in

$$m \ddot{\mathbf{x}} = \sum_{i=1}^4 \hat{\mathbf{x}} \cdot \mathbf{F}_i$$

frame)

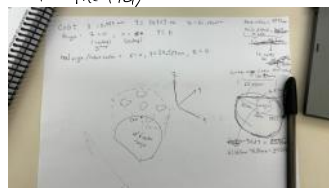
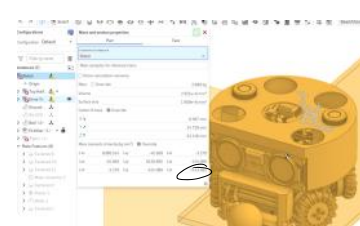
$$m \ddot{\mathbf{y}} = \sum_{i=1}^4 \hat{\mathbf{y}} \cdot \mathbf{F}_i$$

$$I_{zz} \ddot{\phi} = \sum_{i=1}^4 \hat{\mathbf{z}} \cdot (\mathbf{r}_i \times \mathbf{F}_i)$$

The non linear dynamics equations are of the form $\ddot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \dot{\mathbf{x}})$

Solve the system of equations for $\ddot{\mathbf{x}}$, $\ddot{\mathbf{y}}$, $\ddot{\phi}$
 $m = 2.698 \text{ kg}$

I_{zz} is z direction moment of inertia, related to CoG.



$$\text{CoG} = x = 0.097, y = 10.206, z = 61.146$$

$$\text{So } I_{zz} = I_{zz(\text{CoG})} + m(x_{\text{CoG}}^2 + y_{\text{CoG}}^2)$$

$$\text{So } I_{zz}^{\text{robot center}} = I_{zz}(\text{cg}) + m(x_{\text{cg}}^2 + y_{\text{cg}}^2) = 7443.681 + 2.698 \text{ kg} \cdot (0.597^2 + 10.206^2) = 7725.673 \text{ kg m}^2 \cdot \left(\frac{\text{m}}{1000 \text{ mm}}\right)^2 = 0.007725673 \text{ kg m}^2$$

$$\ddot{x} = \frac{\sum_{i=1}^4 \hat{x} \cdot F_i}{2.698 \text{ kg}}$$

$$\ddot{y} = \frac{\sum_{i=1}^4 \hat{y} \cdot F_i}{2.698 \text{ kg}}$$

$$\ddot{\phi} = \frac{\sum_{i=1}^4 \hat{z} \cdot (\hat{r}_i \times F_i)}{7725.673 \text{ kg}}$$

z component of torque

$$\text{State } \dot{X} = \begin{bmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\omega} \end{bmatrix}$$

⑦

integrator.py
- RK4

$$\dot{X} = f(X, u)$$

$$X_k \rightarrow X_{k+1}$$

⑧

controller_pid.py
- PID implementation of controller

$$e = v_{\text{ref}} - v$$

$$u = K_p e + K_i \int e dt + K_d \dot{e}$$

v_x, v_y, ω Controlled

Controller needs to produce wheel commands

$$u = [u_1, u_2, u_3, u_4]^T$$

⑨

controller_mpc.py
-MPC implementation using wheel dynamics
-stub for now, will replace PID later

⑩

simulation.py

initialize state
initialize controller

for t in simulation:

reference = desired motion

wheel_commands = controller(state, reference)

wheel_forces = williams_model(state, wheel_commands)

state_dot = robot_dynamics(state, wheel_forces)

state = RK4(state, state_dot)

save state

save forces

save commands

⑪ Tests

- Check wheel directions

- Check orthonormality where expected (d and a)
- Correct wheel symmetry

12

Simple translate and rotate test in robot frame